

Introduction to Deep Learning

Session 9: Convolutional Neural Networks

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Why Not MLPs for Images?

2. Convolution Operation

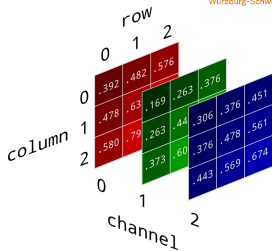
Images in Computers

Image: 3D tensor of shape $H \times W \times C$

- Each entry: pixel intensity, integer 0–255 or float 0–1,
- 2^8 possible values ~ 8 bits
- **Grayscale** ($C = 1$): single channel, 0 = black, 255 = white
- **RGB** ($C = 3$): red, green, blue — standard cameras, screens

Other formats:

- **RGBA** ($C = 4$): RGB + α transparency
- **CMYK** ($C = 4$): print format
- **Medical/hyperspectral**: many channels, often 16-bit depth



DL mostly **RGB and grayscale** — other formats typically converted before use

Image sizes — from benchmarks to real world:

Source	Size	Channels	Pixels
MNIST	28×28	1	784
ImageNet	224×224	3	150 528
Phone camera	4000×3000	3	36 000 000
Full HD screen	1920×1080	3	6 220 800
Medical (CT slice)	512×512	1	262 144

MLP approach: flatten image to vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$, apply fully connected layers

ImageNet: first hidden from $\mathbb{R}^{150\,528} \rightarrow \mathbb{R}^{1024} \Rightarrow \approx 154\text{M}$ parameters — one layer only!

How to handle high-dimensional data efficiently?

How to Look at Images

Human vision:

- Looks for local patterns — edges, corners, textures
- Same pattern recognised anywhere in the visual field
- Simple features combine into complex ones

Technical equivalent:

- **Local connectivity** — connect to small patches only
- **Parameter sharing** — same filter at every location
- **Hierarchical features** — edges → shapes → objects

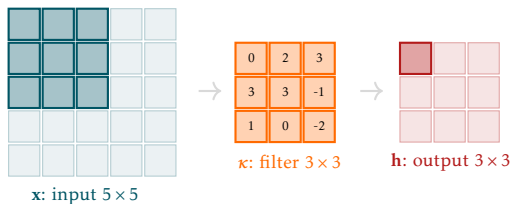
Three key properties: **locality**, **translation invariance**, **parameter efficiency**

These define the **convolutional layer**

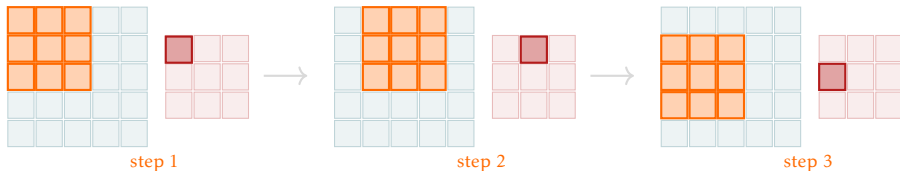
Convolution Operation

Convolution — The Operation

Single step: dot product of local patch and filter $\rightarrow$ one output value



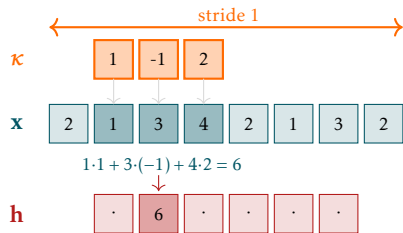
Full operation: same filter slides across input — each position produces one output value



All 3 requirements: local connectivity, parameter sharing, translation invariance

1D Convolution

Filter $\kappa \in \mathbb{R}^k$ slides over signal $\mathbf{x} \in \mathbb{R}^n$ — at each position i , dot product with local patch



$$(\mathbf{x} * \kappa)[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j] \kappa[j]$$

- i — output position; j — index within filter
- Dot product of filter with local patch

Example: at position $i = 1$:

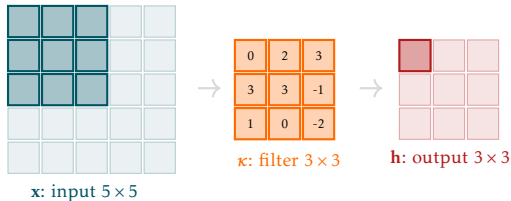
$$1 \cdot 1 + 3 \cdot (-1) + 4 \cdot 2 = 6$$

Convolution — The Math

2D convolution: filter $\mathbf{W} \in \mathbb{R}^{k \times k}$ slides over image $\mathbf{X} \in \mathbb{R}^{H \times W}$

$$(\mathbf{X} * \mathbf{W})[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i + p, j + q] \mathbf{W}[p, q]$$

- At each position (i, j) : dot product between filter and $k \times k$ patch
- Output feature map shape: $(H - k + 1) \times (W - k + 1)$
- Filter weights $\mathbf{W}$ are **learned by backpropagation**



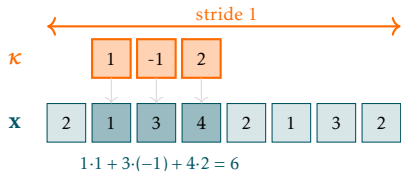
1D Convolution — Intuition

1D convolution: filter $\mathbf{w} \in \mathbb{R}^k$ slides over signal $\mathbf{x} \in \mathbb{R}^n$

$$(x * w)[i] = \sum_{j=0}^{k-1} x[i+j] w[j]$$

- At each position i : dot product between filter and local patch of signal
- Filter detects a specific local pattern
- Same filter applied at every position — **parameter sharing**

Example: edge detector, smoothing filter, derivative approximation

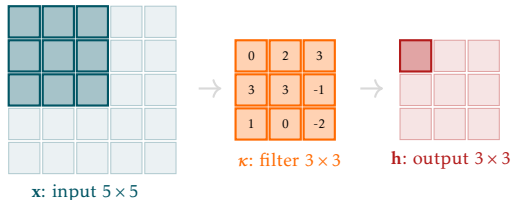


2D Convolution

2D convolution: filter $\mathbf{W} \in \mathbb{R}^{k \times k}$ slides over image $\mathbf{X} \in \mathbb{R}^{H \times W}$

$$(\mathbf{X} * \mathbf{W})[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i + p, j + q] \mathbf{W}[p, q]$$

- At each position (i, j) : dot product between filter and $k \times k$ patch
- Output: **feature map** of shape $(H - k + 1) \times (W - k + 1)$
- Filter learns to detect a specific visual pattern (edge, corner, texture)



Stride and Padding

Stride s : filter moves s pixels at each step

- $s = 1$: standard, output size $\approx$ input size
- $s = 2$: output halved — spatial downsampling
- Reduces computation and memory

Output size: $\lfloor (H - k)/s \rfloor + 1$

Padding p : add zeros around the border

- **Valid** ($p = 0$): no padding, output smaller than input
- **Same** ($p = (k - 1)/2$): output same size as input
- Preserves spatial dimensions

Output size: $\lfloor (H + 2p - k)/s \rfloor + 1$

Multiple Channels and Filters

RGB input: C_{in} input channels — filter must have depth C_{in}

$$(\mathbf{X} * \mathbf{W})[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q] \mathbf{W}[c, p, q]$$

Multiple filters: C_{out} filters produce C_{out} output feature maps

- Each filter detects a different pattern
- Output tensor: $C_{\text{out}} \times H' \times W'$
- Total parameters per layer: $C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$ (with bias)

In PyTorch:

```
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)
```

Receptive field: region of the input that influences a given neuron's output

- Layer 1 (3×3 filter): receptive field = 3×3
- Layer 2 (3×3 filter): receptive field = 5×5
- Layer L (3×3 filters): receptive field = $(2L + 1) \times (2L + 1)$

Key insight: deep networks have large receptive fields despite small local filters

- Early layers detect local features (edges, corners)
- Later layers combine local features into complex patterns (eyes, wheels, faces)
- Hierarchical feature learning — the hallmark of deep networks

Two 3×3 filters have the same receptive field as one 5×5 filter but fewer parameters:
 $2 \times 9 = 18$ vs 25